Module-1

(1) Introduction to Playwright

What is playwright?

→ Open-source automation tool developed by Microsoft (released in 2020)

→ Used for browser automation and end-to-end (E2E) testing

→ Also supports API testing with a dedicated API

→ Built on Node.js (a JavaScript runtime that runs outside the browser).

Key features :-

1. Cross-Browser Support :-
   Works with chromium (Chrome, Edge), Firefox, and WebKit (Safari).

2. Cross-Platform
   Runs on Windows, macOS, and Linux.

3. Multi-language support
   write tests in Java script, Typscript, Java, Python, or C#

4. Mobile web Testing
   supports Chrome (Android) & Safari (iOS)

5. API Testing
   Built-in support for testing backend APIs

6. Auto-Waiting

Automatically waits for elements to be ready before interacting.

7. Handles Complex Elements

Works well with shadow DOM (hard for other tools).

8. Parallel Execution

Runs tests simultaneously in multiple browsers for speed.

9. Built-in Reporting

Supports HTML, JSON, Junit reports + third-party tools like Allure.

Playwright Tools:-

1. Inspector - Debug tests by viewing locators and click points

2. Code Generation (Codegen) - Records user actions and generates test scripts

3. Tracing (Trace viewer) - Captures screenshots, logs steps, and record videos for debugging

API = Application programming Interface

API means Communication b/w two applications

=> Java Script vs. Typescript

Java Script
(Dynamically Typed)

-> No strict type checking
-> Variables can change types

Ex:-

```
let age = 30; // Number
let name = "John"; // string
age = "thirty"; // No error (changes to string.
```

Type Script (Statically Typed)

-> A superset of Javascript (adds static typing)
=> Variables must match their declared type.

Ex:-

```
let age : number = 30; // must stay a number
let name : string = "John";
age = "thirty"; // Error (Type 'string' not
                    assignable to 'number')
```

Why use Typescript?

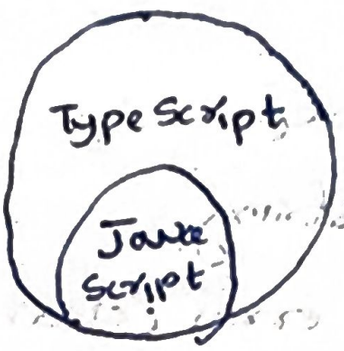
-> ECMAScript (ES) is the standard for JS
=> TS adds extra features (like types) but still compiles to standard JS.
=> Helps catch errors early in development.

2) What is TypeScript?

-> TypeScript is superset of Javascript - it adds extra features to Javascript.

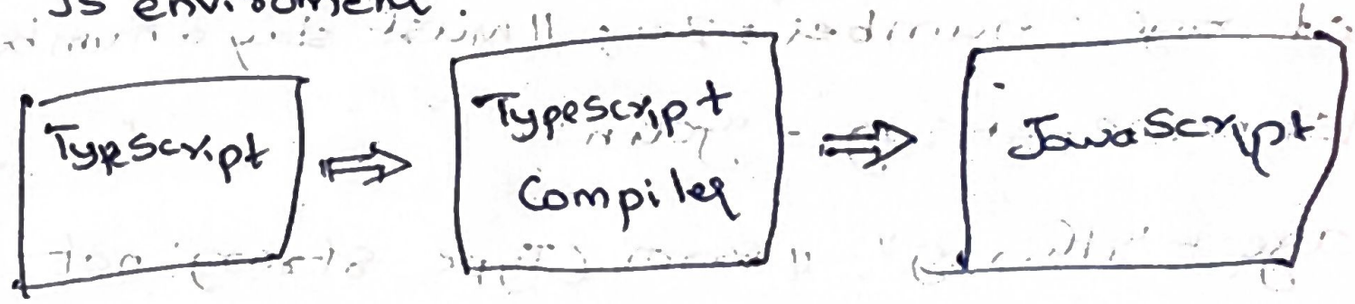
-> It compiles (converts) into regular Javascript so it runs anywhere Javascript does.

-> files end in .ts (instead of .js).



How Typescript works?

1. You write code in Typescript (.ts files).
2. The Typescript compiler converts it into Javascript (.js files).
3. The Javascript runs in browsers, Node.js, & any JS enviroment.



Type script vs. Javascript

-> All Javascript code is valid Typescript!

-> If your Javascript code works, it will work in Typescript too.

-> Typescript just adds optional features (like types) to make coding safer and easier.

Why use Typescript?

-> Helps catch mistakes early (before running the code)
-> Makes large projects easier to manage
-> Works smoothly with existing Javascript code.

Setting up Typescript:-

To begin with Typescript, you'll need these tools:

-> Node.js :- This lets you run the Typescript compiler.
-> Typescript Compiler :- A tool that converts Typescript code into Javascript.
-> VS code :- A free code editor that works great with Typescript.

Quick Summary

-> Node.js -> Runs Typescript
-> Typescript Compiler (tsc) -> Converts .ts files to .js
-> VS code -> Best editor for Typescript
-> Tsx -> Run Typescript without compiling

---

(3) Typescript Variables

-> Variable is a container which can hold data.
-> Variables in Typescript (and Javascript) can be declared using var, let or const
-> Each has different behaviours in terms of scope, value assignment, re-declaration, re-assignment and hoisting

1. Scope :-

Var → function scope

→ Variables declared with var are accessible anywhere inside the function.

→ Can lead to unexpected behaviour because they are not limited to blocks (if, for, etc).

let & const → Block scope

→ Variables are only accessible inside the block {} where they are declared.

→ Safer and more predictable than var.

2. Value Assignment at Declaration :-

var and let - value assignment is not mandatory

const - value assignment is mandatory

3. Re-declaration :-

var - allows re-declaration

let & const - not allows re-declaration

4. Re-assignment :-

Var & let - allows Re-assignment

const - not allows Re-assignment

5. Hoisting (variable access. Before declaration)

Var :- Hoisted but initialized as undefined

let & Const :- Hoisted but not intialized cannot be used before declaration)

Feature	var	let	const
Scope	function	Block	Block
Value Assignment at Declaration	✗	✗ Not mandatory	✓ Mandatory
Re-declare	✓ Allowed	✗ Not allowed	✗
Re-assign / Reinitialization	✓	✓	✗
Hoisting	✓ un-defined	✗ Not initialized	✗
Best use	✗ Avoid	✓ changing values	✓ Constants

Var:-

1. scope - function-scoped (limited to the function where it is declared).
2. Can be redeclared and reinitialized within the same scope
3. Assigning a value at the time of declaration is optional.

let:-

1. scope - Block-scoped (limited to the enclosing {..} block)
2. Cannot be redeclared within the same scope but can be reinitialized.
3. Assigning a value at the time of declaration is optional.

const:-

1. scope - Block-scoped ~~(let)~~ reinitialized
2. Cannot be redeclared within the same scope ~~(let)~~ ~~can be reinitialized~~.

3. Assigning a value at the time of declaration is mandatory.

Best Practices :-

- Avoid var - It can cause unexpected bugs due to function scope
- use let - when a variable needs to change later
- use const - for values that should never change (constants).

Comments

Single line comment → ~~ctrl + /~~

Ctrl + /

// This is a single line comment

Multi-line (Block) comment

Shift + Alt + A

```
/*
 This is a multi-line comment
*/
```

---

⑨ Type Script Data Types

Dynamically Typed vs Statically Typed languages

- JavaScript is a Dynamically Typed language

→ In JavaScript, variable types are checked at runtime, and you can change the type of a variable later.

Ex: let age = 25; // age is a number

age = "twenty-five"; // Now age is a string (no error)

console.log(age); // o/p: "twenty-five"

✓ No errors because Javascript allows type changes dynamically.

Typescript is a statically Typed language

→ In Typescript, variable types are checked at compile time, and you cannot change the type later.

Ex:- let data: number = 10; // data is a number

data: "ten"; // Error :- Type 'string' is not assignable to type 'number'

✗ Type script catches this error before the code runs

Type Safety

⇒ JavaScript is not Type-Safe

⇒ JavaScript allows operations b/w incompatible types, leading to unexpected behaviour.

⇒ Ex:-

const result = "5" + 3; // JavaScript converts 3 to string

console.log(result); // output: "53" (not 8)

⇒ Typescript is Type-Safe

⇒ Typescript prevents operations b/w incompatible types.

Ex:-

```
const result :number = "5" + 3; // Error: Type 'string'
```

is not assignable to type 'number'

✓ Type script warns you about potential bugs early

Key Takeaways:-

→ Dynamic Typing (JS):- Types are flexible (Checked at runtime)

→ Static Typing (TS): Types are fixed (checked at compile time)

→ Type Safety (TS): Prevents operations with wrong types, reducing bugs.

Typescript Types, Annotations & Type Inference

1. Typescript Types (Data Types)

Built-in or custom categories for variables (eg:- number, string, boolean).

Ex:-

```
let isDone: boolean = true; // 'boolean' is Typescript
let score: number = 100; // 'number' is Typescript
```

data type

2. Type Annotations

Explicitly telling Typescript the type of a variable using: type

Ex:- `let name: string = "Alice";` // ': string' → type annotation

`let age: number = 30;` // ': number' → is type annotation

3. Type Inference

Typescript automatically guesses the type if you don't annotate it.

Ex:-

```
let message = "Hello"; // TS infers 'message' as 'string'
let count = 42; // TS infers 'count' as 'number'
//message = 123; // Error (TS knows 'message' must stay a string)
```

Key Differences:

* Type Annotation: You manually define data type
* Type Inference: TS intelligently figures it out

TypeScript Data Types

Primitive Types (Built-in Types)

1. Number
→ Represents both integers and decimals (Ex: 3, 3.14)

2. String
→ Represents text data
→ Can be written in
Single quotes ('Hello')
Double quotes ("Hello")
Backticks (`Hello ${name}`) for template literal

3. Boolean
→ Represents true of false values
→ Used in conditions (e.g., if (isLoggedIn) {...})

4. Null.

→ Represents an intentional empty value

(let x = null).

5. Undefined

→ Represents a variable declared but not assigned (let y; → y is undefined).

6. Any.

→ A flexible type that allows any value (disable Type script checks).

→ Avoid using unless neccessary

(let z: any = "Hello"; z = 10;).

7. Union Type

→ Allows multiple types for a variable

(let id: string | number = "123";).

8. Void

→ Used for functions that don't return anything (function log(): void {

console.log("Hi"); })

Non-Primitive types (Objects & Custom types)

1. Arrays 3. Tuple 5. functions

2. Class 4. Interface

Key Takeaways::

-> Primitive types (numbers, string, boolean, etc.) are hold single values.

-> Non-primitive types (Array, Class, Interface, etc.) are complex and hold structured data.

-> Avoid any when possible - use proper types for better code safety.

-> Union types(|) allows flexibility when a variable can be multiple types.

---

(5) TypeScript Operators

1. Arithmetic Operators

These operators perform basic mathematical operations

-> + (Addition): Adds two numbers -> 10+5 = 15

-> - (Sub) -> * (Multiplication) -> / (Division)

-> % (Modulus) Remainder

-> ** (Exponentiation) Raises a number to a power

2. Assignment operators

Used to assign values to variables

-> += -> x+1 = 5 (x = x+5)

-> -= -> (x = x-5)

-> *= -> (x = x*5)

-> /= -> (x = x/5)

-> %= -> (x = x%5)

3. Increment & Decrement Operators

Used to increase or decrease a value by 1

→ ++ (Increment)

1. x++ (Post. increment: 1st use the value, then increase it)

2. ++x (Pre-increment: 1st increase, then use the value)

→ -- (Decrement)

1. y-- (Post-decrement: 1st use the value, then decrease it)

2. --y (Pre-decrement: 1st decrease, then use the value)

4. Relational / Comparison Operators

Used to compare values and return true or false.

→ <, >, <=, >=, ==

→ == (Equality check, only compares value)

Ex: 10 == "10"

→ != (Not Equal)

→ === (Strict equality, compares both value and type)

→ !== (Strict in equality)

5. Logical Operators

Used to combine multiple conditions

-> && (AND) -> Returns true if both conditions are true

Ex:- (x>5 && x<15) -> true

-> || (OR) -> Returns true if atleast one condition is true

Ex:- (x>10 || y<5) -> true

-> ! (NOT) -> Reverse the condition (T -> F, F -> T)

Ex:- !(x>5) -> false

6. Ternary Operator (Conditional Operator)

-> A shortcut for if-else.

Syntax:-

condition ? value_if_true : value_if_false;

---

(6) Conditional Statements

Conditional statements in TypeScript help in decision-making based on conditions.

Below are the main types:

1. if statement

-> The if statement executes a block of code only if a specified conditions is true.

-> If the condition evaluates to false, the code inside the if block is skipped

Syntax:- if (condition) {

// statement

}

2. if-else statement

→ The if-else statement executes one block of code if the condition is true, and another block if the condition is false.

Syntax:-
```
if (condition) {
    // stat
} else {
    // stat
}
```

3. Nested if-else (if-else if) statement

→ This is used when multiple conditions need to be checked sequentially.

→ The first true condition is executed, and the rest are skipped.

Syntax:-
```
if (cond 1) {
    // stat
} else if (condt 2) {
    // stat
} else if (condt 3) {
    // stat
} else {
    // stat
}
```

4. Switch-case Statement

-> The switch statement allows testing a variable against multiple values (case)

-> If a match is found, that case block executes. The break statement stops execution after a match.

=> The default case runs if no match is found.

Syntax:

```
switch(expression){
    case value 1:
        // statement
        break;
    Case value 2:
        // statement
        break;
    case value 3:
        // statement
        break;
    default:
        //stat
}
```

Key takeaways:-

1. if -> Executes code if only one condition is true
2. if-else -> Executes one block for true, another for false
3. if-else-if -> checks multiple conditions sequentially
4. switch-case -> Compares a value against multiple cases for efficiency.

(7) Loops in TypeScript

Looping statements in TypeScript allow executing a block of code multiple times based on a condition. Below are the corrected syntax and examples for each loop.

1. **While Loop:-**

The while loop executes a block of code as long as the condition is true

Syntax:-
```
while (condition){
    // Code to execute
}
```

2. **Dowhile Loop:-**

The do-while loop executes the code at least once before checking the condition.

Syntax:-
```
do{
    //Code to execute
}while (condition);
```

3. **for loop:-**

The for loop is useful when the no. of iterations is known

Syntax:-
```
for(initialization; condition; Inc/Dec){
    // Code to execute
}
```

4. **Break statement:-**

The break statement stops the loop immediately when a condition is met.

3. Continue statement

The continue statement skips the current iteration and moves to the next one.

Conclusion:-

=> Use while when you want to check the condition before execution.

=> Use do-while when you need the loop to run at least once

=> use for when you know how many times the loop should run.

=> use break to stop a loop early

=> use continue to skip an iteration and proceed to the next one.

---

(8) functions (Part 1)

1. Named functions:-

A named function has a specific name and can be reused multiple times in the code.

Syntax:- function functionName(parameters): return type

{

// function body

}

2. Anonymous functions:

An anonymous function does not have a name. It is usually assigned to a variable.

Syntax:-

```
let variableName = function(parameters): returnType
{
// function body
}
```

3. Arrow functions (Lambda functions)

Arrow functions provide a shorter syntax for writing functions.

Syntax:-

```
let functionName = (parameters): returnType => exp;
```

Key points:-

> Uses => (fat arrow) instead of function keyword

> Single-line functions don't need {} or return keyword

> Multi-line functions require {} and return

Summary Table:-

Type	Syntax Ex	Key features
Named	function sum(a,b) { return a+b; }	has a name, reusable, traditional syntax
Anonymous	let multiply = function(x,y){ return x*y;};	No name, stored in variable
Arrow	let square = (x) => x*x;	shorter syntax, uses =>

Callback and Overbaded functions in Typescript

In typescript functions can have special features like callback functions and function overloading making them more powerful and flexible.

Callback functions:-

A callback function that is passed an argument to another function and gets executed later

function Overbading in Typescript

function Overbading allows you to define multiple versions of a function with the same name but different parameters or return types

Basic rules for function overloading...

1. Define Overload Signatures
2. function implemention
3. function call back

Correct fn overloading
① Different paramet type
② Different no of paramet
③ Different return type

④ Optional parameter
↓
2

⑤ Rest parameter
↓
...

⑥ Callback function
↓
Passed an argument & executed later

⑦ function Overloading
↓
Multiple Signatures for implem

Quick Summary:-

① Named function
↓
Has function Name

② Anonymous function
↓
No function Name

③ Arrow function
↓
Uses =>

(10) Arrays in TypeScript

TypeScript Arrays :-

An array in TypeScript is a special type of variable that can store multiple values. These values can be of the same type or a combination of different types.

Declaration of an Array :-

Arrays in Typescript can be declared in two main ways.

1. Using square brackets ([]) (array literal syntax)
2. Using Array<Type> (generic array syntax).

Approach 1 : Array literal

```
let names : string[] = [];
```

Approach 2 : Generic Array Syntax

```
let names : Array<string> = [];
```

Accessing Array Elements :-

- Access by index (indexing starts from 0).
- Printing arrays using console.log().

Iterating over an Array :-

There are multiple ways to loop through an array.

Using a for loop :-

```
for (let i = 0; i < empNames.length; i++) {
    console.log(empNames[i]);
}
```

Using for... in loop (indexes)

```
for (let i in empIds) {
  console.log(empIds[i]); // i represents index
}
```

Using for... of loop (values)

```
for (let element of data) {
  console.log(element); // 'element' represents actual array values
}
```

Tuple : A special Type of Array :-

→ A tuple is a fixed length array where each element has a specific type.

It helps in storing multiple fields of different data types together.

---

(11) Array Methods part (1)

1. push() :-

Adds one of more elements to the end of an array.

Syntax:- array.push(element1, element2,..)

2. pop()

Removes the last element from an array and returns it.

Syntax:- array.pop()

3. shift()

Removes the 1st element from an array and returns it. Syntax:- array.shift()

4. unshift():
- Adds one of more elements to the beginning of an array.
Syntax: array.unshift(element1, element2,..)

5. concat():
Combines two of more arrays and returns a new array.
Syntax: array.concat(value1, value2,..)

6. slice():
Extracts a section of an array without modifying the original array.
Syntax: array.slice(startIndex, endIndex)
Note: endIndex is optional

7. splice():
- Adds or removes elements at any position in the array
Syntax: array.splice(start, deleteCount, item1, item2,..)

8. indexOf():
Returns the index of the 1st occurrence of a value or -1 if not found.
Syntax: array.indexOf(value, fromIndex?)

9. includes():
checks if an array contains a specific value. Returns true or false.
Syntax: array.includes(value, fromIndex?)

10. toString()

Converts the array to a comma-separated string

syntax:- array.toString()

(13) Array Methods part (2)

Quick Revision:-

Method	Purpose
push()	Add at end
pop()	Remove last
shift()	Remove first
unshift()	add ~~first~~ at beginning
concat()	Combine arrays
slice()	copy part (original array unchanged)
splice()	Add / Remove / Replace (original array changes)
indexOf()	find index (-1 if not found)
includes()	Returns true or false
toString()	Convert array to string

=) Difference b/w slice() & splice()

slice()

↓

Does NOT modify original array

splice()

↓

Modifies original array

(12) Arrays ~~part~~ Methods part (2)

-> Advanced array methods

1. forEach():-

Executes a function for every element
It doesn't create a new array.

Syntax:- array.forEach((currentvalue, index, array) => {})

Uses:- Printing, Logging. Performing an action

2. map() :-

Creates a new array.
Every element is modified.

Syntax: array.map((currentvalue, index, array) =>

=> map() -> transforms every element -> returns new array

3. filter():-

selects elements based on a condition.

=> filter() -> keeps only matching elements.

=> Difference b/w map() and filter()

map() - returns same no. of elements
filter() - may return fewer elements

4. reduce() :-

Converts an array into one value

⇒ reduce() → many values → one value

5. some() :-

checks whether at least one element satisfies the condition.

⇒ some() → one is enough

6. every() :-

checks whether all elements satisfy the condition

⇒ every() → Everyone should pass

Comparison Table

Method	Returns	Main Purpose
forEach()	Nothing(void)	Perform an action for each element
map()	New Array	Transform every element
filter()	New Array	Select matching elements
reduce()	Single value	Sum, total, count, etc.
some()	Boolean	At least one matches
every()	Boolean	All must match

(13) Strings in Typescript

A string in Typescript is a sequence of characters used to represent text. It can be declared using:

1. single quotes 'like this'
2. double quotes "like this"
3. Backticks (Template literals) `like this` - useful when embedding variable inside strings.

☆ A string is a sequence (combination) of characters enclosed in quotes.

Backticks?

Backticks allow template literals, which support string interpolation using ${}.

String Methods

1. .length → Get the no. of characters
2. .toUpperCase() / .toLowerCase() → change case
3. .charAt(index) / .indexOf(substring) → Get a character or find a substring
4. .substring(start, end) → Extract a part of the str
5. .includes(substring) → check if a substring exist
6. .startsWith() / .endsWith() - check start or end of string
7. .replace(old, new) -- Replace part of a string

5. .split [illegible]

9. .trim() / .trimStart() / .trimEnd() → Remove extra spaces. Removes spaces from start, and of both

String Immutability:-

Strings are immutable → once created, they can't be changed. Methods returns new strings

Multi-line strings:-

Using backticks allows strings to span multiple lines

Quick summary:-

Method	Purpose
length	Number of characters
toUpperCase()	Uppercase
toLowerCase()	Lowercase
charAt()	character at index
indexOf()	Position of text (-1 if not found)
substring()	Extract part of string
includes()	check if text exists
startsWith()	check beginning
endsWith()	check ending
replace()	Replace first occurence
split()	Convert string→array
trim()	Remove spaces (both sides)
timesStart()	Remove leading spaces (start)
trimEnd()	Remove trailing spaces (end)
~~time~~) concat()	Join strings

(14) Objects

What is an object?

An object is a collection of key-value pairs

It contains:

→ properties (variables) - e.g., name, age, salary

→ Methods (function) - e.g., getDetails(), setDetails()

Objects represent real-world entities like Employee, student, Product, etc.,

Accessing properties::

- Dot notation → employee.name
- Bracket notation → employee["name"]

Modifying::

employee.job = "Manager";

Different ways to create objects in TS/JS :-

1. Using object Type
2. Inline Type object
3. Using type aliases
4. Using classes

(18) class. Read only Properties, optional Properties, static properties and Methods in Typescript

class :- A class is a blueprint for creating objects with properties and methods.

Read-only properties :-

-> Read only properties cannot be changed after initialization.

-> use the readonly keyword

-> useful for constants or values that shouldn't be modified.

optional Properties :-

-> optional properties are not required when creating an object.

-> Defined using the ? symbol

-> useful when a property is not always needed

Static Properties and Methods :-

-> static members belong to the class, not the instances.

-> Accessed using the class name, not through objects.

-> Useful for utility methods of shared data

(16) Inheritance, super(), and Access Modifiers.

Inheritance in Typescript:-

→ Inheritance allows a class (child) to acquire properties and methods from another class (parent)

→ Use the extends keyword

Super():-

→ Super() is used in the child class constructor to call the parent class constructor.

Method overriding:-

→ A child class can redefine (override) a method from the parent class.

→ The method in the child class should have the same name and signature.

Access Modifiers:-

Used to control the visbility of class members (properties and methods).

1. Public -> Accessible from anywhere (default)
2. private -> Accessible only within the same class.
3. ~~protected~~
3. protected -> Accessible in the class and its subclasses

Interface and Modules

What is an Interface?

An interface in Typescript is a way to define the structure of an object. It tells the compiler what properties and types an object should have. It's like a blue print for objects

Interfaces help in:-

-> Ensuring type safety

-> Making code more readable and maintainable.

-> Supporting object-oriented programming.

Syntax:-

```
interface InterfaceName {
  property1 : type;
  property2 : type;
}
```

Summary:-

-> Interface define the structure of objects

-> They support optional, readonly, and function properties.

-> Interfaces can be extended for reusability.

-> They improve code clarity and maintainability.

Differences B/w class and Interface in TypeScript

Feature	Class	Interface
Definition	Blueprint to create objects with implementation.	Defines structure / type of an object with no implementation
Contains	Properties, constructors methods (with implementation)	only property and method signatures (no implementation)
Instantiation	Can be instantiated using new keyword	cannot be instantiated directly
Inheritance	supports single inheritance using extends	supports multiple inheritance using extends.
Implements	A class can implement one or more interfaces	Interface cannot implements a class.
Modifiers	Support access modifiers (public, private, protected)	Does not support access modifiers

Summary:-

->Use interface when you want to define the structure of contract

->Use class when you want to define implementation and create objects.

Modules

What are Modules?

Modules in Typescript are a way to organize and encapsulate code. They help in dividing large programs into smaller, manageable files and allows reusability by exporting and importing functionalities (like variables, functions, classes, etc.)

Why use Modules?

-> Code reusability

-> Better maintainability

-> Avoid name conflicts using scoped declarations

How to create and use Modules?

->Use the export keyword to expose code from a file (module).

->Use the import keyword to bring in code from another file.
